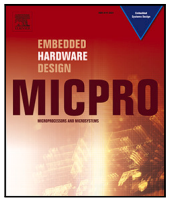
Microprocessors and Microsystems 107 (2024) 105047

Contents lists available at ScienceDirect

Microprocessors and Microsystems

journal homepage: www.elsevier.com/locate/micpro



Hardware accelerated Active Noise Cancellation system using Haar wavelets

P. Santos [a,b,*], E. Mendes [a], J. Carvalho [a,b], F. Alves [c], J. Azevedo [d], J. Cabral [a,e]

[a] Algoritmi Centre - University of Minho, Guimaraes, 4800-058, Braga, Portugal
[b] DTx - Digital Transformation CoLab, Guimaraes, 4800-058, Braga, Portugal
[c] International Iberian Nanotechnology Laboratory, Braga, 4715-330, Braga, Portugal
[d] Bosch Car Multimedia Portugal, S.A., Braga, 4705-820, Braga, Portugal
[e] CEiiA - Centro de Engenharia e Desenvolvimento de Produto, Matosinhos, 4450-099, Porto, Portugal

ARTICLE INFO

Keywords:
Active Noise Cancellation
Least Mean Square
Filtered-x Least Mean Squared
Wavelet
Haar
FPGA

ABSTRACT

Active Noise Cancellation (*ANC*) systems are widely used to mitigate unwanted noises in several applications, such as automotive environments and high-end headsets. Multi-Channel (*MC*) *ANC* systems have shown promise in creating improved silent zones. Typically, these systems are implemented on *FPGA* platforms due to the systolic nature and granularity of optimization of these devices. This article describes the design, implementation, and evaluation of a wavelet-based *MC ANC* Filtered-x Normalized Least Mean Square (*FxNLMS*) on an *FPGA* platform.

The use of wavelet transform enables the decomposition of complex noise signals into spectrally more compact signals (i.e., easier to process). In this work, for each decomposed signal, an independent *NLMS* is applied. The system implements 64 parallel *NLMS* with 1000 coefficients. Additionally, the static *FIR* filters employed for secondary and tertiary path estimations are of the 2047th order. The system adopts an integer arithmetic architecture and operates at a sampling rate of 47.97 kHz. To assess the performance of the wavelet-based approach, benchmark tests were conducted by comparing it against a similar implementation without the wavelet transform. The evaluation was performed using noise reduction (*NR*) tests with spectrally rich (20 Hz to 10 kHz) and high dynamic range noises. The experimental setup involved two error microphones and two secondary sources.

The results show that the wavelet-based version has overall better performance than the traditional implementation, particularly in the higher frequency band of the spectrum (1 kHz to 8 kHz). For instance, in the case of city ambient noise (a realistic noise with high dynamic range), the relative *NR* achieved was 8.23 dB.

To the authors' knowledge, this is the first time that the implementation and field-test of a wavelet-based *MC ANC* on an *FPGA* platform was conducted. Moreover, the obtained results show that the novel approach is better in reducing complex noises than the traditional implementation – without wavelets.

1. Introduction

An Active Noise Cancellation (*ANC*) system is used to reduce unwanted noises, either periodic or random [1]. There are many applications for these systems, ranging from headphones (for aviation or general usage [2]) to the automotive sector. The latter adopts it to eliminate stationary noises originating from the engine, road, or the external environment [3].

The working principle consists of a transducer, usually a speaker that reproduces a cancellation signal, which is a sound wave with the same amplitude as the signal to be cancelled, but with an inverted phase. The two sound waves (the signal to be cancelled and the cancellation signal) combine to form a new wave, in a process called interference, resulting in destructive interference as the two signals cancel each other out [1].

These systems have strict temporal requirements to ensure the causality of the system. The first implementations on embedded devices were on Digital Signal Processors (*DSPs*) mainly due to their lower cost. To create bigger silent zones, the *ANC* must increase the channel count, which consequently increases the complexity. This motif pushed the adoption of *FPGA*-based platforms. Among other reasons, these platforms are more flexible due to their systolic nature and have a higher granularity of optimization, which are crucial aspects to deal with the increase in complexity without jeopardizing the causality of the multi-channel *ANC* systems.

* Corresponding author at: Algoritmi Centre - University of Minho, Guimaraes, 4800-058, Braga, Portugal.
E-mail addresses: id7524@alunos.uminho.pt (P. Santos), jcabral@dei.uminho.pt (J. Cabral).

https://doi.org/10.1016/j.micpro.2024.105047
Received 14 September 2022; Received in revised form 31 July 2023; Accepted 2 April 2024
Available online 5 April 2024

The elimination of random noises (such as music or human speech) is a long-standing ambition. Currently, the Multi-Channel (*MC*) *ANC* implementation on embedded systems is often optimized for lower frequencies. To increase the actuation bandwidth, the *ANC*'s algorithm can be complemented with the wavelet transform [4]. Wavelets are notoriously able to handle non-stationary signals, meaning that their translation-scale spectrum decays faster when compared with the windowed Fourier transform [4,5]. Since each *ANC* slot now operates on a smaller frequency band, the overall system achieves better performance. There are several works in the literature describing wavelet-based *ANC* systems [6–11]. These are mainly implemented and tested in computer simulators or in controlled environments. Moreover, the implementations of *MC ANC* systems on *FPGA*-based platforms are scarce.

A scalable architecture for *FPGA* of an *MC ANC* was presented in [12]. It implements the feedback Filtered-x Least Mean Squared (*FxLMS*) algorithm with step size normalization and leakage factor. The system used fixed-point arithmetic to save resources. A 2×2 configuration was tested with the filter's order of 2048th. The sampling frequency was 160 kHz. The noise reduction of broadband noise with a limited bandwidth (200 Hz to 600 Hz) was 12 dB. Another *FPGA*-based *MC ANC* system with high dimension (24×24) was described in [13]. It implements a feedforward *FxLMS* with a parallelizable architecture that allowed to save resources. The filter's order was 199th, and the sampling frequency was 25 kHz. The system was tested on a modified window. Broadband noise (500 Hz to 2 kHz) was attenuated by 10.1 dB. The work described in [14] presented an *MC ANC* for *FPGA* with $1 \times 18 \times 1$ (18 error microphones). The filter's order was 400th, and the system's processing time was 32.44 µs. The work showed that the *FPGA* implementation, which utilized integer arithmetic, had comparable results to the software version of the system that utilized floating-point architecture.

Our work presents an *FPGA*-based *MC ANC* 2×2 system. The implementation encompasses the Filtered-x Normalized Least Mean Squared (*FxNLMS*) algorithm with wavelets. To evaluate the system's performance, we conducted a comparative study between the wavelet-based implementation and the conventional approach without wavelet transform, commonly employed in the literature. Several types of complex noises (spectrally rich and with high dynamic range) were tested. We observed that the *MC ANC* system with wavelets was able to perform noise reduction on the low band (from 20 Hz to 1 kHz) and high band (from 1 kHz to 8 kHz). Whereas the performance, on the high band, of the implementation without wavelets, was more dependent on the noise's specific characteristics. In our understanding, this is the first time an *MC ANC* system using wavelet transform was described, implemented, and tested on an *FPGA* platform. The system has 64 parallel wavelet channels (i.e., 64 parallel *NLMS* algorithms running), each with adaptive filters with 1000 coefficients. The static filters (used for the secondary and tertiary path estimations) were configured with 2048 coefficients, and the sampling frequency of the system was 47.97 kHz. To save resources, fixed-point arithmetic was used, as well as many optimization tweaks that are described throughout the document.

The remainder of the manuscript is organized as follows. Section 2 describes the *FxLMS* algorithm. Section 3 describes the wavelet transform and its implementation with a filter bank. Section 4 presents the proposed *MC ANC* system. Section 5 explains the *FPGA* implementation of the *MC ANC* and describes in greater detail its main modules. Section 6 describes the benchmark performed to the *MC ANC* system with wavelets against the implementation without wavelets. It mainly encompasses noise reduction tests but also an explanation of the hardware resourced utilized. Finally, Section 7 draws the conclusions.

2. Filtered-x least mean squared

In this *ANC* system, an adaptive filter called Filtered-x Least Mean Squared (*FxLMS* [15]) was used. The core of this algorithm is the Least Mean Square (*LMS* [16–19]) method. Eq. (1) is used to calculate the coefficients ($\hat{\boldsymbol{w}}$) for the *LMS FIR* filter, which approximates the response of the primary path (from the noise source to the error microphone, see Fig. 1) to minimize the error signal (*e[n]*). Where $\hat{\boldsymbol{w}}[n]$ is the current *LMS FIR* filter coefficient vector, μ is the step size, $\boldsymbol{xs}[n]$ is the delayed signal reference, *e[n]* is the error signal, T denotes matrix transpose, and $\hat{\boldsymbol{w}}[n+1]$ is the *LMS FIR* filter coefficients vector for the new iteration. The error signal results from the sum of the noise signal (*d[n]*) with the anti-phase signal (*y'[n]*) (see Eq. (2)). The *FIR* filter output signal, *y[n]*, is the outcome of transforming the reference signal ($\boldsymbol{x}[n]$) accordingly to the *LMS*'s calculated coefficients, as described by Eq. (3). Then, the anti-phase signal is obtained by multiplying the filter output signal (*y[n]*) by −1. The error signal will converge to zero as the *FIR* filter response converges to that of the primary path.

The *LMS* algorithm has the limitation of requiring virtually no delay between the emitted anti-phase signal and the input of the coefficients update algorithm. This path is commonly referred to in the literature as a secondary path. For that reason, Widrow et al. [15] proposed the Filtered-x Least Mean Square to account for the secondary path's delays. By considering the effect of the secondary path on the system, the algorithm can correctly calculate the coefficients that minimize the error. For this reason, the same delay must be created in the reference signal that feeds the coefficients update algorithm.

$$\hat{\boldsymbol{w}}[n+1] = \hat{\boldsymbol{w}}[n] + \mu \boldsymbol{xs}[n] e^T[n] \tag{1}$$

$$e[n] = d[n] - y'[n] \tag{2}$$

$$y[n] = \hat{\boldsymbol{w}}^T \boldsymbol{x}[n] \tag{3}$$

The adaptive filter used in this work is a variant of the *LMS*. The latter is susceptible to the input power changes of the reference signal; for example, a sudden volume change may turn the algorithm unstable [20]. To solve this problem, one of the possible solutions was the normalization of the reference signal before using it to calculate the coefficients. Using this method, called Normalized Least Mean Square (*NLMS*), the estimation of the learning rate is much more effective, which allows the error to converge to zero. Eq. (4) is the formula for updating the *NMLS* coefficients.

$$\hat{\boldsymbol{w}}[n+1] = \hat{\boldsymbol{w}}[n] + \frac{\mu}{\|\boldsymbol{xs}[n]\|^2} \boldsymbol{xs}[n] e^T[n] \tag{4}$$

3. Wavelet transform

The wavelet transform is a signal processing technique that computes the signal spectrum over time, like the windowed Fourier transform. However, the wavelet transform uses a compact support basis function called wavelet. This compact support makes them particularly suitable to handle non-stationary signals. There are several wavelet-based functions, such as Haar (see Eq. (5)) and Daubechies [21]. In this work, the Haar wavelet basis function was selected due to its binary nature.

$$\Psi(t) = \begin{cases} 1 & if\ 0 \le t < 1/2 \\ -1 & if\ 1/2 \le t < 1 \\ 0 & otherwise \end{cases} \tag{5}$$

The wavelet function (Ψ) is scaled and translated (represented as s and u in Eq. (6)) before being convoluted with the signal of interest *f(t)* (see Eq. (7)). This translation and scaling in wavelet transform plays similar roles to time and frequency in the windowed Fourier transform.

Unlike the windowed Fourier transform that has the same resolution for all time–frequency boxes, the wavelet transform has different *time–frequency* (translation-scale) resolutions (Eq. (6)), making it more suitable to analyze the frequency-temporal dynamics of the signal of interest.

$$\Psi_{u,s}(t) = \frac{1}{\sqrt{s}} \Psi\left(\frac{t-u}{s}\right) \tag{6}$$

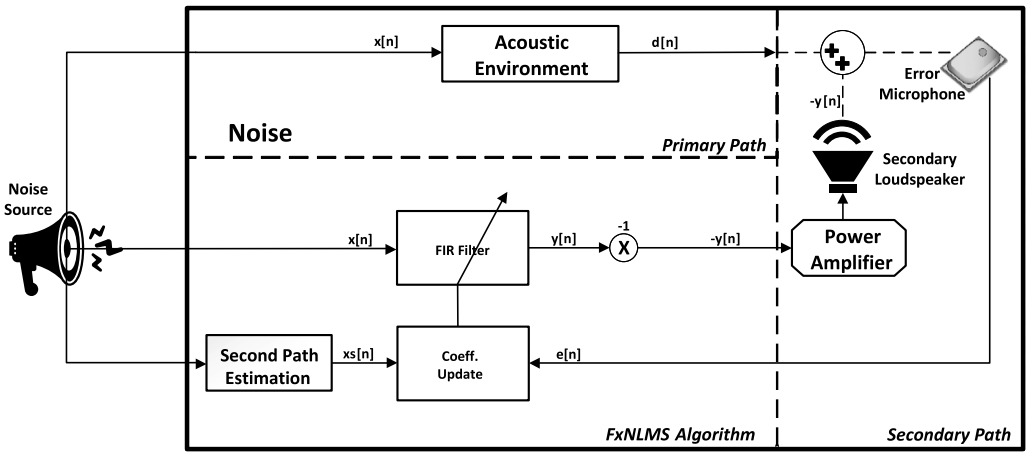


Fig. 1. *FxLMS* block diagram.

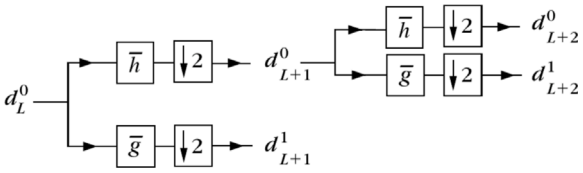


Fig. 2. Dyadic wavelet transform.

$$W f(u,s) = \int_{-\infty}^{+\infty} f(t)\Psi_{u,s}(t)dt \tag{7}$$

In this work, the wavelet transform implementation is done with banks of filters. The filter banks' technique [22–24] decomposes a discrete signal (e.g., *f[n]*) into two half-sized signals, using a filtering and subsampling procedure. This procedure is cascaded as shown in Fig. 2, where $\overline{h}$ is a low-pass filter and $\overline{g}$ is a high-pass filter, d^0 is the wavelet approximation coefficient, and d^1 is the wavelet detail coefficient.

Fig. 2 illustrates the Dyadic Wavelet Transform (*DWT*) in which the wavelet approximation coefficient is cascaded as opposed to the wavelet detail coefficient. This wavelet transform results in translation-scale boxes illustrated in Fig. 3 (higher temporal resolution at high *frequencies*, but with low spectral resolution; and low temporal resolution at low *frequencies*, but with high spectral resolution). Fig. 4 depicts a Wavelet Packet Transform (*WPT*) where both the approximation coefficient and the detail coefficient of the wavelet are decomposed, resulting in translation-scale boxes with the same resolution.

In this article, we implemented a variant of the *WPT* called Undecimated Wavelet Packet Transform (*UWPT* [25]). As the name suggests, this approach does not perform decimation after filtering; instead, the wavelet's filter coefficients are upsampled.

Using this technique, we double the wavelet coefficients at each level of decomposition, as opposed to the *WPT*, where the number of wavelet coefficients is equal to the number of samples of the input signal. Fig. 5 depicts the *UWPT* where the dilated filters, $\overline{h}$ and $\overline{g}$, are upsampled.

Eqs. (8) and (9) represents the filter upsampling process by inserting zeros between each pair of coefficients of $\overline{h}(l)$ and $\overline{g}(l)$, where j is the decomposition level, and L is the total number of coefficients of the original filters $\overline{h}(l)$ and $\overline{g}(l)$.

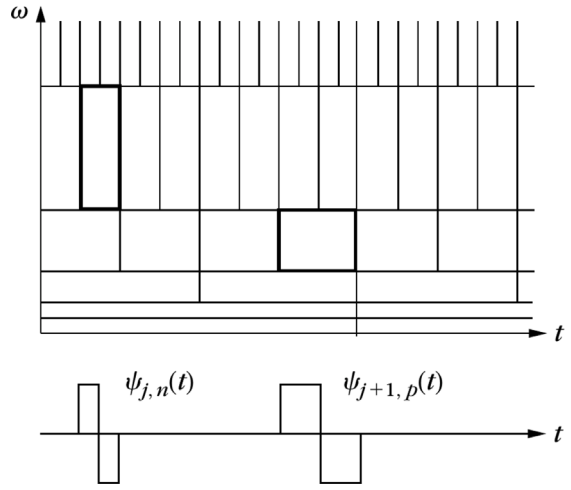


Fig. 3. Translation-scale boxes in Gabor space [4].

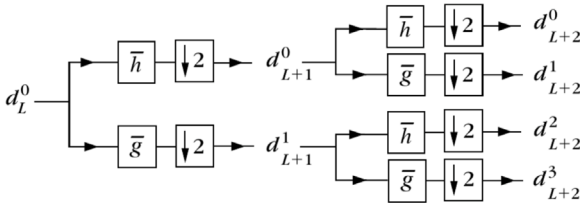


Fig. 4. Packet wavelet transform.

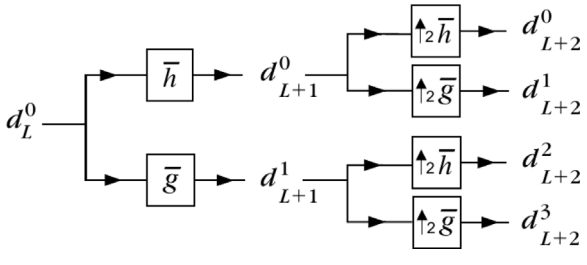


Fig. 5. Undecimated wavelet packet transform.

$$\overline{h}_j[l] = \left[\overline{h}[0], \underbrace{0,\ldots,0}_{2^j-1}, \overline{h}[1], \underbrace{0,\ldots,0}_{2^j-1}, \ldots, \overline{h}[L-1], \underbrace{0,\ldots,0}_{2^j-1}\right] \tag{8}$$

$$\overline{g}_j[l] = \left[\overline{g}[0], \underbrace{0,\ldots,0}_{2^j-1}, \overline{g}[1], \underbrace{0,\ldots,0}_{2^j-1}, \ldots, \overline{g}[L-1], \underbrace{0,\ldots,0}_{2^j-1}\right] \tag{9}$$

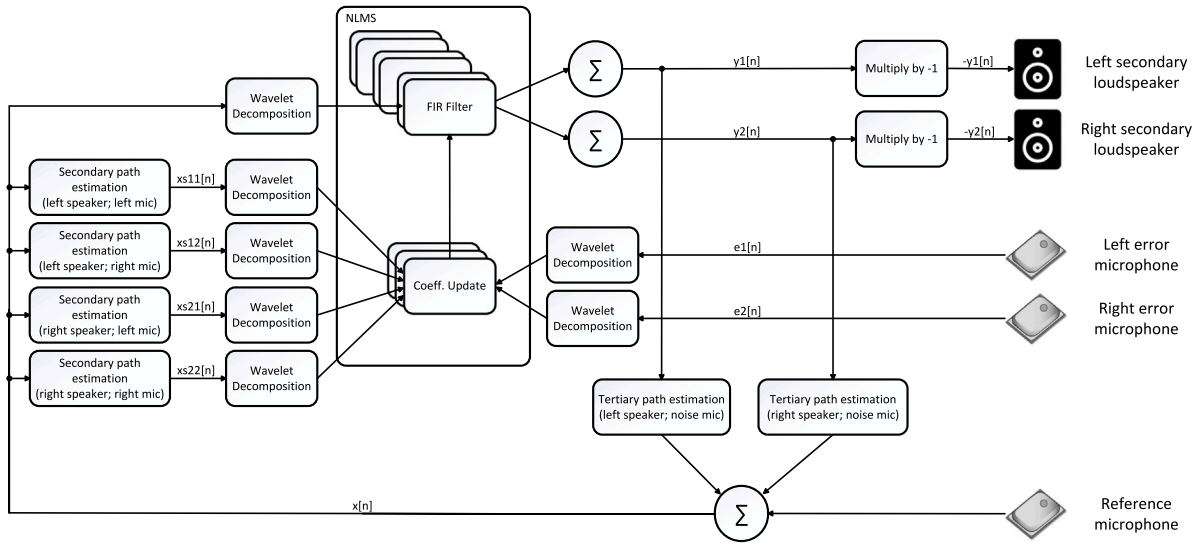


Fig. 6. *ANC* block diagram with wavelets.

As already mentioned, we decided to implement the *UWPT* rather than the *DWT* or *PWT*. These techniques have drawbacks that limit their usage. As an example, the *DWT* has two limitations: (i) the output rates on each output channel are different and; (ii) there is an output channel that contains half of the spectrum of the input signal (i.e., higher frequencies). For that reason, the *FxLMS* algorithm will operate with a high bandwidth input, and consequently, this would result in lower performance. For the *PWT*, the drawback is the low output sampling rate. The *FxLMS* algorithm requires an adequate sampling rate of the input signal (which is related to the update of the coefficients). This sampling rate is particularly important to actuate on the higher spectral components of the input signal. A lower sampling rate means that the *FxLMS* will not be able to cope with such fast-transitioning signals. For those reasons, the *UWPT* was the selected wavelet implementation. It has all outputs with the same sample rate and with the same rate as the input signal. Also, the input signal's bandwidth is equally divided by all outputs. In this way, the *FxNLMS* algorithm is constantly fed with an adequate sampling rate, thus improving its overall performance.

4. Proposed wavelet-based multi-channel *ANC* system

The present section describes the *MC ANC* system and the arrangement of the various constituent elements. It consists of a 2×2 configuration (two secondary loudspeakers and two error microphones). The positioning of the elements was crucial for the overall performance of the *ANC* system.

4.1. Multi-channel ANC system overview

The block diagram depicted in Fig. 6 represents a *MC* approach, with two error microphones, two secondary loudspeakers, and one reference microphone.

The error ($\boldsymbol{e[n]}$) and reference ($x[n]$) signals are processed and divided into several channels using *UWPT*. Each channel is an input for a dedicated *NLMS* algorithm. Four *FIR* filters apply the secondary path estimation to the reference signal, one for each secondary loudspeaker and error microphone combination. In addition, there are two other *FIR* filters to apply the third path estimation (the path from the secondary loudspeakers to the reference microphone) to the anti-phase signal ($\boldsymbol{y[n]}$). The goal is to eliminate the presence of the anti-phase signal on the reference signal (as the reference microphone shares the same space as the secondary loudspeakers) and obtain a signal that contains almost only information from the noise source. Finally, the outputs of all *NLMS* algorithms are added and then shifted to obtain the anti-phase signal.

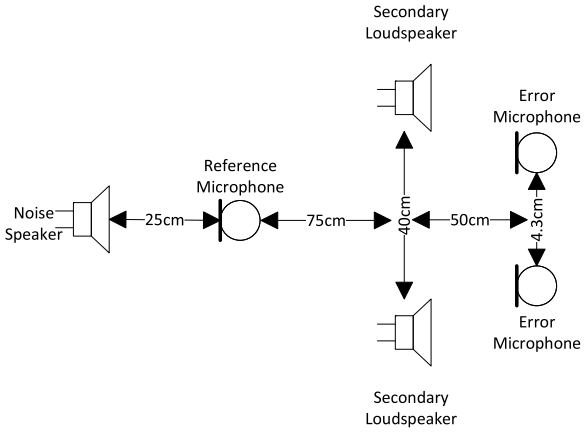


Fig. 7. *ANC* components arrangement.

There is an *NLMS* algorithm for each channel of the *UWPT*, and each one processes a sub-band of the input signal. The number of *NLMS* modules running in parallel influences the system's efficiency. The higher the number of modules running, the better the cancellation of the noise will be. However, since each reference signal channel has its unique power, it is necessary to normalize each channel independently. In this way, all error signals from all channels converge to zero at almost the same rate.

4.2. Arrangement for random noise cancellation

One way to interpret the *FxNLMS* algorithm is to bear in mind that its objective is to constantly search for the coefficients (of the *FIR* filter) that estimate the response of the primary path. Nevertheless, it is not always possible to find a physically realistic solution. For example, the delay of the primary path could be smaller than the delay of the secondary path. Such a particular solution would not represent a typical geometry of an *ANC* system. To improve the stability of the system, the constituent components were arranged, as illustrated in Fig. 7. The delay of the primary path (the distance from the reference microphone to the error microphone) is larger than the secondary path (distance from the secondary loudspeaker to the error microphone).

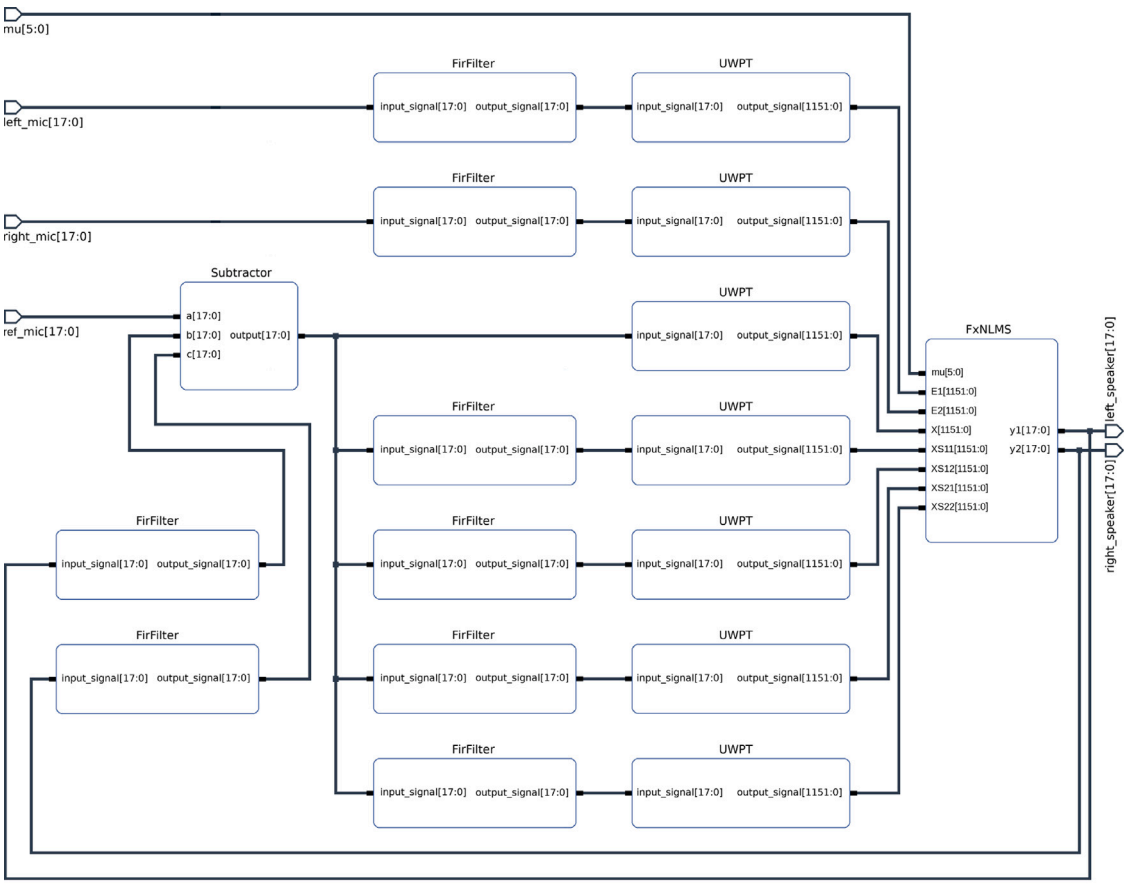


Fig. 8. *FPGA* implementation block diagram.

5. Implementation of the wavelet-based multi-channel *ANC*

The first step was the selection of an *FPGA* that had enough resources to implement this *ANC* system. Furthermore, these resources must be enough to accommodate all the *NLMS* modules needed to obtain good results in reducing random noise, as previously mentioned in Section 4. Having this in mind, the greater the number of *NLMS* modules running in parallel, the better the random noise cancellation will be, the *XILINX FPGA XC7Z045* was chosen. This *FPGA* is one of the ones with a considerable number of resources available on the market in their family. The implementation of the *ANC* system has been divided into different modules (see Fig. 8). Here we will focus on the three main ones, namely the *FIR* filters, the *UWPT* and the *NLMS*. As the system's performance depends on the amount of parallel *NLMS* algorithms, the modules were implemented with consideration for the *FPGA*'s logic resources.

5.1. FIR filter module

The *FIR* filter implements the mathematical convolution operation. The filter has 2048 coefficients of 18-bit. Its values were pre-processed (during the environment's impulse response detection) and stored on a dedicated *BRAM*. The input signal is received and acquired via the *input_signal* port (18-bit), and the incoming samples are also stored in a *BRAM*. This *BRAM* is managed as a circular buffer. Each new sample is stored in the next address, and when it gets full, the first stored sample will be rewritten with the newest sample.

The optimization of resources was considered. The module operates at the system clock of 116.667 MHz. Compared to the input signal's sample rate (47.97 kHz), the large system clock allowed the multiplications of the convolution operation to be done serially (i.e., pipeline style). Because of that, only one *DSP* was needed to perform all the multiplications. For each incoming sample, the module has 2432 clock cycles to perform all the calculations and takes roughly 2048 clock cycles (2048 coefficients/multiplications). Finally, the division operation of the convolution was performed by bit-shifting the final result. We divided the result by a power of two value, which was 2048. The choice for the total number of coefficients was also linked to this idea; 2048 is the power of two value nearest and inferior to 2432.

5.2. UWPT module

As was mentioned in Section 3, the transform is implemented by a cascade of filters. There are n outputs on the cascade, and each output is composed of m levels of filters (see Fig. 5). In this implementation, six levels ($m = 6$) of filters were used. Instead of having a set of filters, exploring the nature of the Haar wavelet basis function allowed us to convolute the six levels into a single filter. In the end, the whole cascade was represented by 64 independent filters (see Fig. 9). Their coefficients were precalculated and stored into the *BRAM*s through initialization files. The input signal (18-bit) is being filtered in parallel by the filters, and in the output, the module creates 64 channels, working at the same sample rate as the input signal. Each of these channels is a spectral sub-band of the input signal.

Each channel has 64 coefficients (1-bit) that are stored in a *BRAM*. The acquisition and storage of the input signal are made similarly, as explained for the *FIR* filter. The *BRAM*, for the input signal, has 64 cells of 18-bit.

The convolution operation in this module does not perform any multiplications. In turn, the nature of the Haar wavelet is explored to perform only additions and subtractions, thus excluding the need to use

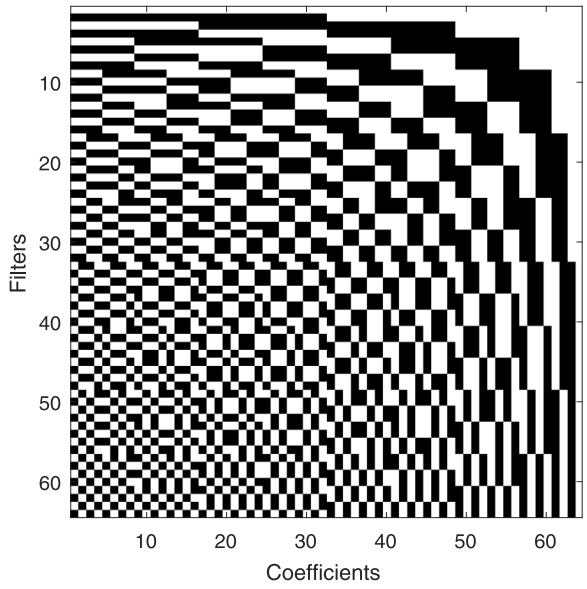


Fig. 9. Wavelet Haar module coefficients (white square means 1 and black square means −1).

*DSP*s. This way, if the coefficient is 1, an addition is performed; if the coefficient is −1, a subtraction is done.

Finally, as in the *FIR* filter module, the division is made by simply bit-shifting.

5.3. NLMS module

The *NLMS* module is composed of two main blocks, the *FIR*s and the coefficients update.

In the first stage at the coefficients update block, there is a normalization operation. This normalization was crafted to avoid mathematical divisions, thus saving *DSP* resources (details can be found later in this section). Each *UWPT* outputs 64 channels and each channel is processed by a single *NLMS* algorithm. As such, there are 64 *NLMS*, and since we have a *MC* configuration, we need two *FIR* filters for each *NLMS* algorithm (one for each canceling speaker). In total, 128 *FIR*s blocks were instantiated.

Each *FIR* filter has 1000 coefficients of 48-bit. The number of coefficients was limited to the available resources of the *FPGA*. The implementation of *FIR* filters inside the *NLMS* module is similar to the one described in the *FIR* filter module section. The coefficients are stored in *BRAM*s as well as the last 1000 samples of the input signal. The storage of the input signal is done as described in the *FIR* filter module section. As the *FPGA*'s *DSP*s have *18 + 25* bit inputs and 48-bit outputs each and the coefficients are 48-bit, we need three *DSP*s per filter.

The *NLMS* coefficients update block implements Eq. (4). The algorithm starts by calculating the noise signal energy, $\|\boldsymbol{xs}[n]\|^2$, of the last 1000 samples. This is done by adding all the elements after being multiplied by themselves (see Eq. (10)). For this operation, we use the same strategy as described in the *FIR* filter module section (the multiplications are done serially to use the minimum *DSP*s as possible).

$$\|\boldsymbol{xs}[n]\|^2 = \sum \boldsymbol{xs}[n]^2 \tag{10}$$

After calculating the energy of the ***xs[n]*** signal, it is necessary to invert the result. To avoid the division operation, the order of the bits was reversed around the decimal point, followed by a shift of one bit to the left, as shown in Table 1. Then, all bits to the left of the first 1, counting from right to left, are set to be zero. In this way, we get a very rough approximation of the inverse of a number.

Table 1
Computing a very rough approximation to the inverse of a number by reversing the order of the bits.

Number		Inverse	
Decimal	Binary	Binary	Decimal
1	00000001.0000000	00000001.0000000	1.0000000
2	00000010.0000000	00000000.1000000	0.5000000
4	00000100.0000000	00000000.0100000	0.2500000
8	00001000.0000000	00000000.0010000	0.1250000
16	00010000.0000000	00000000.0001000	0.0625000
32	00100000.0000000	00000000.0000100	0.0312500
64	01000000.0000000	00000000.0000010	0.0156250
100	01100100.0000000	00000000.0000010	0.0156250

Once the inverse calculation of the ***xs[n]*** signal energy is done, the next step is to multiply it by the constant μ (see Eq. (4)), which is always a power of two. This multiplication is done by shifting $log_2(\mu)$ bits to the left.

The rest of the calculations are done serially to use as few *DSP*s as possible.

5.4. UWPT module implementation

The wavelet module decomposes an input signal's spectrum in 64 translation-scale sub-bands. Consequently, there are 64 parallel filters to process the input signal (See Fig. 10), each composed by an arithmetic unit (containing an adder and a subtractor) and an accumulator. The accumulator is represented by the Flip-Flop (*FF*) connected to the arithmetic unit output. The reason for having the convolution operation done without any multiplier (i.e., *DSP*) is linked to the nature of the Haar wavelet, as explained in Section 5.2. Like the implementation of the *FIR* filter described in Section 5.1, the filtering is done at a rate much higher (2432 times higher) than the acquisition frequency, allowing to perform the convolution operation sequentially.

The arithmetic unit has two operand inputs, the input signal and the feed output of the accumulator, and one control line, the *SEL* bit. The first operand input, containing the input signal, is 18-bit wide, and since we are doing 64 additions or subtractions, we need six extra bits in the accumulator to store the result. This explains why the second operand input is 24-bit wide. The operation to be performed (addition or subtraction) is selected by the *SEL* bit of the arithmetic unit, which is also connected to the output data port of the *BRAM_A*, where each bit represents the value of a given coefficient for the associated filter. The final result of the filter is *truncated* at the 18 most significant bits.

In the *BRAM_A* are stored the coefficients of the 64 filters, having 1-bit wide word, being its content depicted in Fig. 9. As already mentioned, the output of the *BRAM_A* is directly connected to the *SEL* bit of the arithmetic unit. The access to the *BRAM*'s content is done at the system clock, being the read address port (*ADDR_RD*) managed by the six least significative bits of the *Counter_A*, enough to address the 64 coefficients.

The acquisition of the input signal (18-bit) happens when the *Counter_A* (incrementing at the system clock) crosses the 0 value. A *FF* fetches the input signal data to be stored on the *BRAM_B*, being this *BRAM* managed as a circular buffer. The write address port (*ADDR_WR*) is updated at the input sampling frequency and stores the incoming samples at the *Counter_B* value minus one address. Note that this counter is also working at the input signal's frequency, 47.97 kHz. Whenever the end of the *BRAM_B* is reached, the incoming sample overwrites the oldest stored sample. Reading samples from the *BRAM_B* is done through the read address port (*ADDR_RD*). The *Counter_B* value is used as a reference for the first value to be read (write address value plus one). To the reference, an offset is added, which is related to the sequential nature of the filtering process. Since the reading is done at the system clock, all the samples stored in memory are read from the oldest to the newest (in this order) for the filtering process.

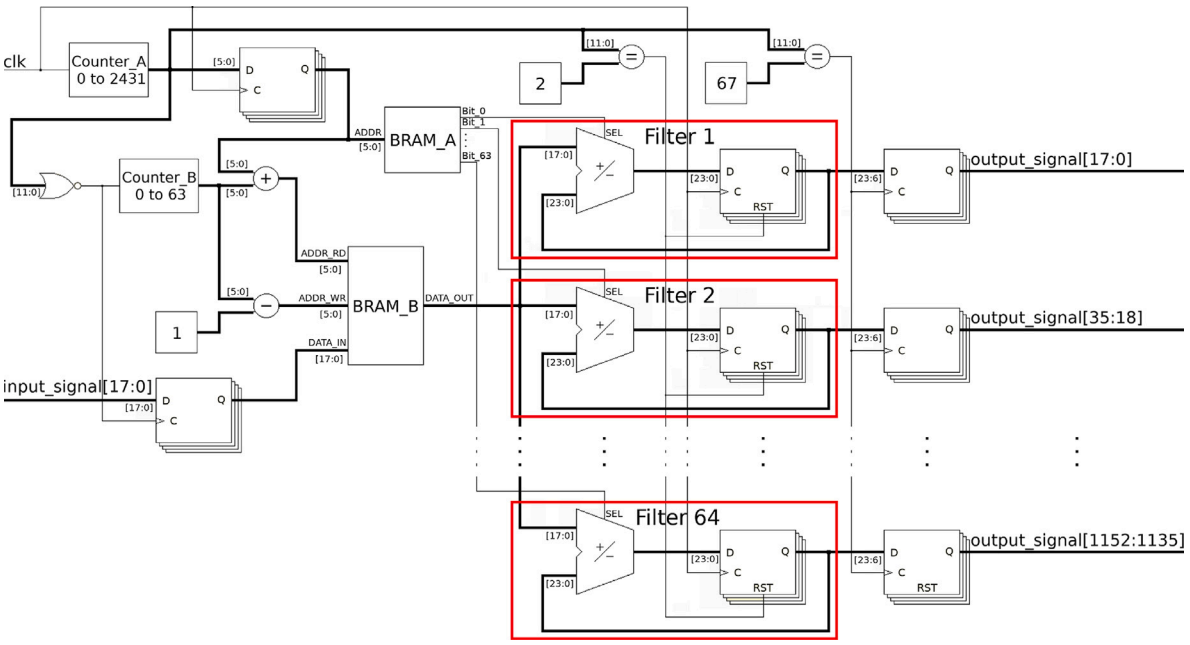


Fig. 10. *Wavelet Haar* module implementation diagram.

On the filters accumulator *FF*, there is a reset signal. This is needed for the convolution of each new input sample. Moreover, the *BRAM*s elements work as a pipeline system, but two clock cycles are required to output the memory requested content. For this reason, the reset signal of the accumulator *FF* is set when *Counter_A* is equal to 2. This delay and the internal delay of 1 clock cycle of the accumulator *FF* result in a final clock count of 67. It takes 67 system clocks to obtain a processed sample.

5.5. Second and third path impulse responses

The impulse responses of the second and third paths (see Fig. 1 and Section 4.1, respectively) are calculated in the Processing System (*PS*). First, the *PS* plays a wave file on the left secondary loudspeaker that contains a pre-recorded swept-frequency sine. At the same time, the signals captured by all three microphones, the noise microphone and the two error microphones, are recorded. Next, the same procedure is repeated for the right secondary loudspeaker, and three more captured microphone signals are stored. Then, the *PS* calculates all the six impulse responses using the reproduced signal and the six recorded signals. Finally, it sends all six impulse responses to the respective *FIR* filter modules on the Programmable Logic (*PL*) side.

The procedure described here, in this sub-section, is the first operation that the *ANC* system does when it is turned on.

6. System evaluation

To perform the evaluation of the *MC ANC FxNLMS* system with wavelets, a similar version of the system was implemented without wavelet's transform. This type of implementation represents the traditional approach adopted by most *ANC* systems found in the literature, serving as a basis for comparison with the novel approach proposed in this study. The first part of the evaluation involves a noise reduction comparison between the two implementations. Subsequently, a comprehensive description of the computational resources utilized by both approaches is provided.

6.1. Noise reduction tests

6.1.1. Methodology

Fig. 11 shows a test overview. On the left side of the image, the arrangement of the components (explained in Section 4.2) are shown, and on the right side of the image are the interface of the components with the system. It is depicted an *MC ANC* with two error microphones and two secondary loudspeakers. Note that the error microphones are both on the same circular *PCB* and are separated by 4.3 cm. In contrast, the reference microphone's *PCB* only has one *MEMS* microphone.

MEMS microphones (*MP34DT01* from *STMicroelectronics*) were used for signal acquisition (both reference signal and error signals), having an acoustic overload point of 120 dB_{SPL}. The output signal of these microphones is a *PDM* signal. Consequently, a *PDM* to *PCM* converter was developed to convert the 1-bit signal (sampled at 3.070 MHz) into an 18-bit word (sampled at 47.97 kHz). For the signal reproduction, which includes the anti-phase signals and even the noise signal, the loudspeakers *SC-M40* from *DENON* were used. These are normal range loudspeakers with an impedance of 6 Ohm. To drive the secondary loudspeakers, the power amplifiers FDA450LV (class *D*) from *STMicroelectronics* were used, featuring an I^2S audio interface. Note that the noise source is totally independent of the *MC ANC* system. The noise loudspeaker was connected through the *FH-10W* amplifier (*SONY*) and subsequently connected to a Personal Computer for playback of the noise recordings.

The *MC ANC FxNLMS* 2×2 system, with wavelets and without wavelets, possessed 1000 coefficients ($\hat{\boldsymbol{w}}$) for the adaptive filter. For the six static *FIR* filters (used on the secondary and tertiary path estimation), the number of coefficients was 2048. Note that, in the case of the version with wavelets, the system has 64 parallel *NLMS* adaptive filters. The sampling frequency of the system was 47.97 kHz. The value for the step size, μ, of both implementations, was chosen empirically to achieve the best possible results. For the case of the implementation with wavelets, the step size was 2^{-14}, and for the version without wavelets, it was 2^{-10}.

The tests were conducted in the academic auditorium, which is an amphitheater with 482 seating places. It is a spacious environment

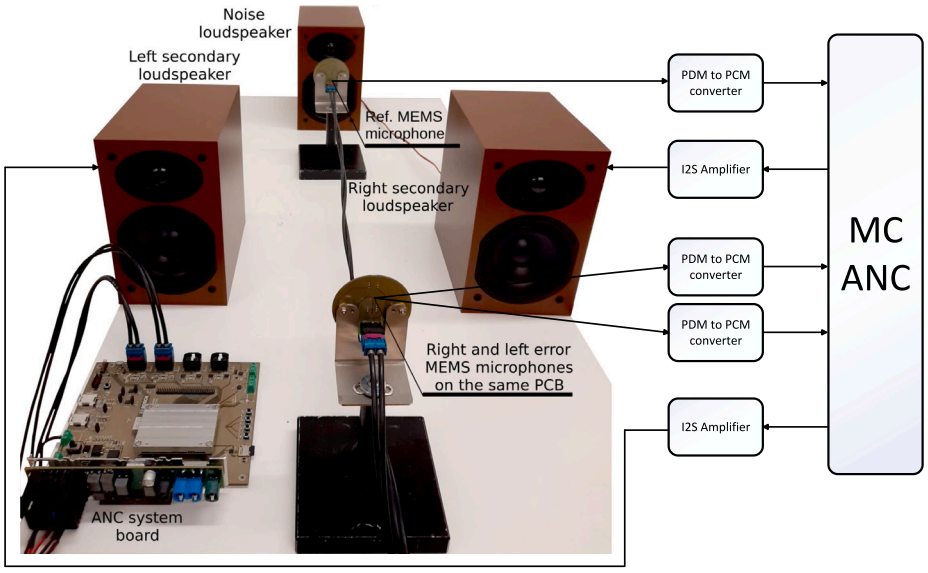


Fig. 11. *FPGA ANC* system hardware.

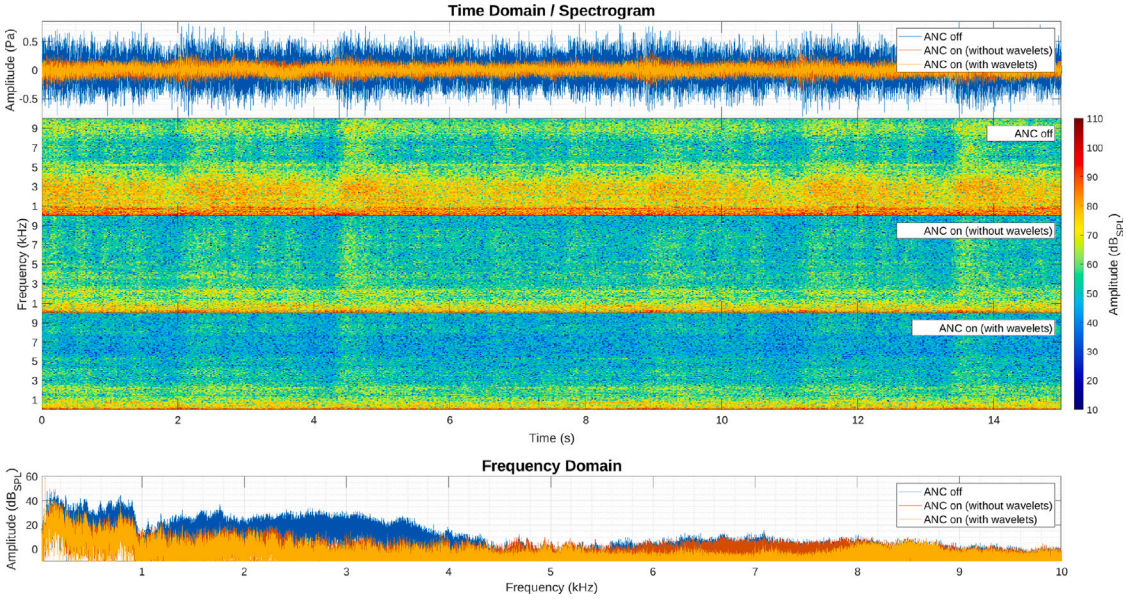


Fig. 12. Experimental results of the proposed *ANC* system - classical music as a "noise" source. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

designed for hosting conferences, congresses, cinema screenings, and performing arts events. The auditorium features a wooden floor stage measuring 10 × 13 m, on which the tests were performed. As a source of "noise", four signals were selected to evaluate the system's performance: *O Fortuna*, a piece of classical music, an urban street environment sound captured in a busy city, an ambient noise of a crowded restaurant, and pink noise. *O Fortuna* was chosen due to its rich spectrum and high dynamic range, suitable for showing the performance of this *ANC* system. Urban street and crowded restaurant ambient noises were selected because they are unpredictable and are more realistic use cases. The pink noise is random and has a full spectrum that fades along with the frequency, which is suitable for evaluating the system's maximum actuation frequency.

The results were obtained using an independent monitor microphone setup. This setup employed the *GRAS 146AE 1/2″* free-field microphone as the transducing element, placed in close proximity to the left-error microphone. This microphone offers a frequency response from a few Hz to 20 kHz, a suitable dynamic range, and a sensitivity of 50 mV/Pa. To power the microphone, the *12AK* power module from *GRAS* was used. The analog signal was then digitally converted using the *NI USB-6356 ADC* (*National Instruments*). Subsequently, the signal was acquired and processed in *MATLAB 2023a* with the assistance of the software package *Analog Input Recorder* (sampling frequency of 200 kHz).

6.1.2. Results

The results were obtained with the monitor microphone during 15 s. The graphics for all results (Figs. 12–15) were organized as follows. The first graphic (top) depicts the noise (time representation) captured without active cancellation (the blue curve when *ANC* is *off*). The orange and yellow signals are the results with active cancellation from the *ANC* system without wavelets and with wavelets, respectively. Note that the temporal signals were time-shifted so that they became overlaid. The second, third, and fourth graphics (central graphics of the

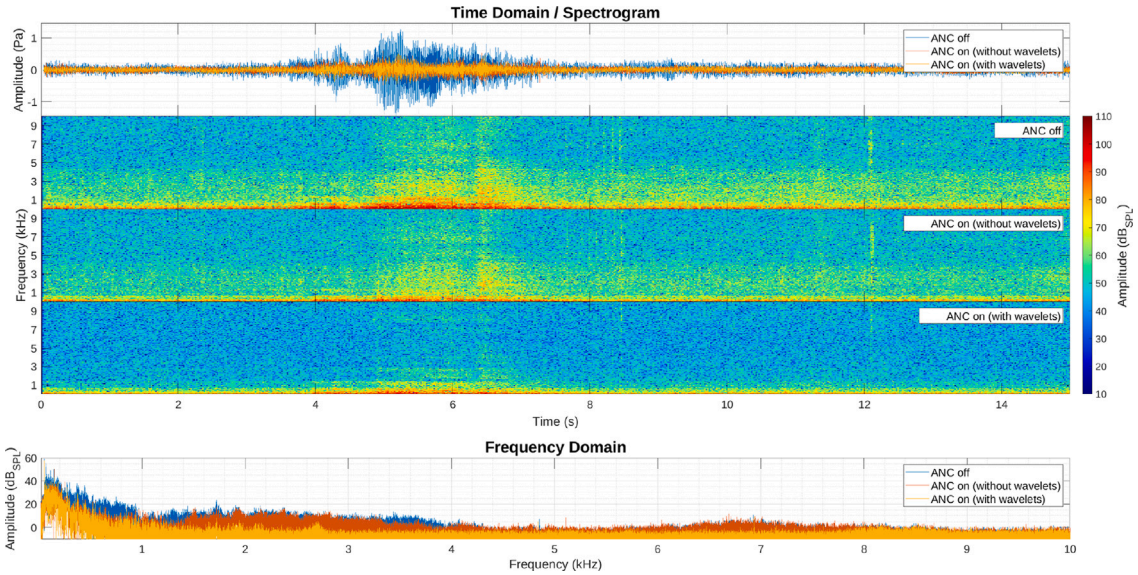


Fig. 13. Experimental results of the proposed *ANC* system - city ambient sound as a noise source. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

figure) are spectrograms of the time-domain signals. The amplitude is in dB_{SPL}, and the frequency is in kHz. To draw the spectrograms, *WPT* up to level 9 with the Daubechies wavelet having two vanishing moments (*db2*) was used. This allowed a good translate-scale resolution, with the scale ordered by its frequency dominance. Lastly, the fifth graphic shows the noise spectrum (*FFT*) when the *ANC* system is *off* (blue curve) and when the *ANC* system is *on*, without wavelets (orange curve), and with wavelets (yellow curve). The amplitude is in dB_{SPL}, and the frequency is in kHz. The calculation of the Noise Reduction (*NR*) was done using these signals.

Fig. 12 shows the results obtained when the "noise" source is the classical piece, *O Fortuna*. The *NR* of the *ANC* system without wavelets was 8.13 dB, and the NR with the wavelets' version was 9.70 dB. The central spectrograms depict the spectral noise evolution over time. It is possible to observe when the system is *off* and when the system is *on* without wavelets and with wavelets, respectively. As can be seen, the system without wavelets has a limited ability to reduce noise across the whole spectrum, whereas the implementation with wavelets was able to reduce an entire band and work with the same performance in low and high amplitude noise. The bottom graphic shows the *FFT* of the time-domain curves for their entire duration. The implementation without wavelets could reduce the noise frequencies up to approximately 4.5 kHz. Moreover, the implementation with wavelets was able to reduce the noise up to 8 kHz.

Fig. 13 depicts the results for the urban environment noise signal. In this case, the attenuation gain was 8.14 dB for the implementation without wavelets and 8.89 dB for the implementation with wavelets. Considering the time-domain signals, the noise's attenuation is noticeable when the *ANC* system is working with both implementations. Nevertheless, when regarding the spectrograms, it is possible to observe an attenuation in the whole spectrum, more pronounced with the implementation with wavelets. At the 5th to 7th second, the sound of a car engine was heard, and it is visible that the implementation with wavelets attenuated this high-intensity sound. Looking at the *FFT*, the system without wavelets was able to reduce noise frequencies up to 1 kHz. However, the version with wavelets reduced almost all frequencies up to 8 kHz (consistent with the previous result).

Fig. 14 illustrates the quotidian sound of a crowded restaurant. The trend from the previous results remains present for the case of implementation with wavelets. There is a noticeable reduction of the noise, 8.55 dB for the version with wavelets. For the version without wavelets, the reduction was 7.79 dB. The spectrograms show an attenuation for the wavelets' implementation on a larger bandwidth (up to 8 kHz), compared with the implementation without wavelets that could reduce noise frequencies up to 4 kHz.

The last result (Fig. 15) addresses the pink noise. By observing the time-domain signals, it can be concluded that the attenuation of the noise existed, but with less impact compared to the above-discussed results. The average attenuation, in this case, was 7.46 dB for the implementation without wavelets and 8.32 dB for the implementation with wavelets. Considering the spectrograms, it is possible to observe that the differences in actuation between both implementations are minimal. More detailed information can be obtained by inspecting the *FFT* graphic. Both implementations reduced almost all frequencies up to 8 kHz. However, the implementation with wavelets was slightly better.

6.1.3. Discussion

Comparison between implementations

In Table 2 are the *NR* results obtained for all noise types, for the implementations with and without wavelets. Results were obtained through the monitor and error microphones (left and right). They were calculated for the whole signal's spectrum (20 Hz to 10 kHz), for the low band (20 Hz to 1 kHz) and high band (1 kHz to 10 kHz). The goal of dividing the results into low and high band is to show more clearly the differences between both implementations.

From Table 2, was elaborated the Table 3. This table presents the relative *NR* of the implementation with wavelets compared to the implementation without wavelets.

The following discussion considers the monitor microphone's results. The wavelet's implementation had a satisfactory performance in all noise types (see Table 2). On average, the results were 8.41 dB and 12.40 dB for the low band and high band, respectively. The implementation without wavelets was mainly consistent in the low band (where the noise's intensity was higher). Nevertheless, on the high band, some variability is observed (see Table 2, in particular the classical and city noises). On average, the results were 7.56 dB and 8.07 dB for the low band and high band, respectively. Thus, the main differences between both implementations (without wavelets and with

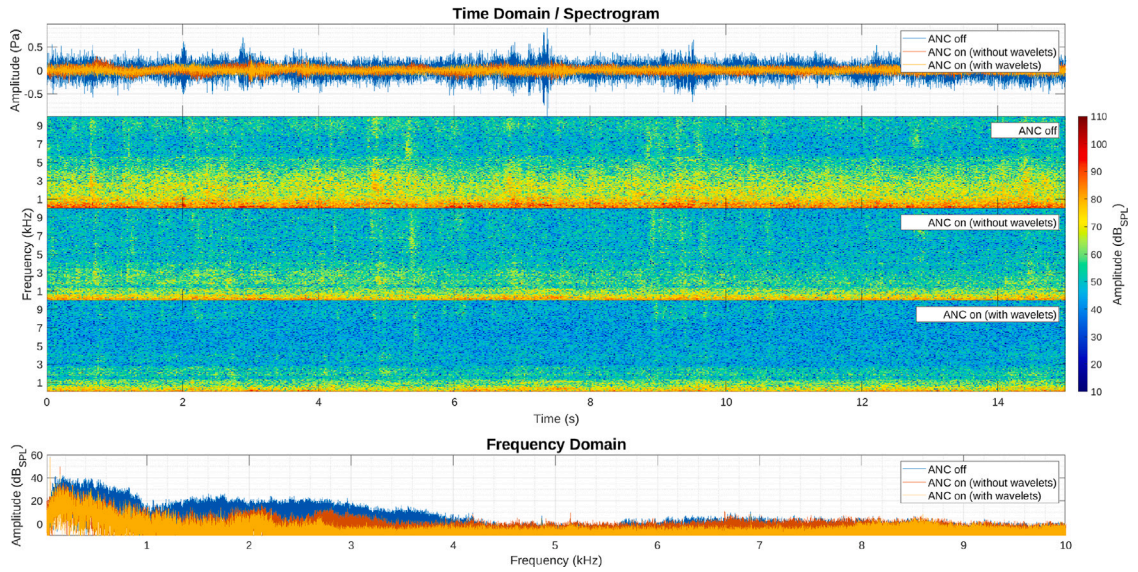


Fig. 14. Experimental results of the proposed *ANC* system - crowded restaurant ambient sound as a noise source. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

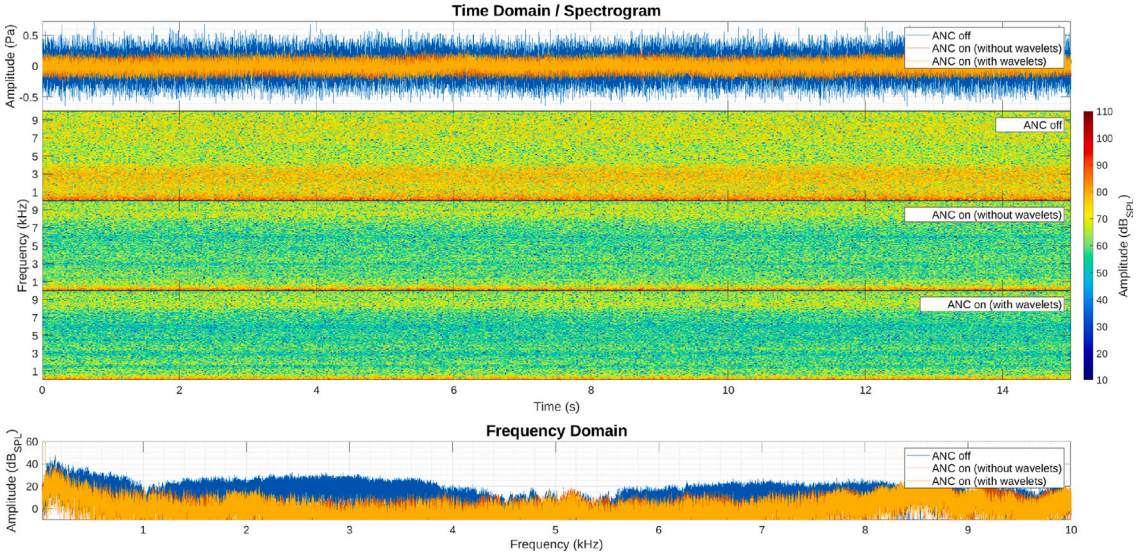


Fig. 15. Experimental results of the proposed *ANC* system - pink noise as a noise source. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

Table 2
Average NR for several noise types of both ANC's implementations (without and with wavelets). All spectrum (20 Hz to 10 kHz), low band (20 Hz to 1 kHz) and high band (1 kHz to 10 kHz).

ANC	Mic.	Band	Noise type/NR (dB)			
			Classical music	City noise	Restaurant noise	Pink noise
w/out wavelets	Monitor	All	8.13	8.14	7.79	7.46
		Low	7.47	8.28	7.73	6.74
		High	13.04	1.71	8.70	8.83
	Error (L/R)	All	8.24/8.34	8.02/8.03	7.65/7.73	4.86/6.74
		Low	7.81/7.86	8.12/8.15	7.66/7.77	6.57/6.55
		High	11.36/11.34	1.97/1.70	7.55/7.30	3.47/6.91
w/Wavelets	Monitor	All	9.70	8.89	8.55	8.32
		Low	8.92	8.88	8.33	7.50
		High	16.61	9.94	13.06	9.98
	Error (L/R)	All	10.62/10.83	9.51/9.59	9.91/10.07	6.49/8.32
		Low	10.12/10.21	9.51/9.58	9.84/9.94	8.46/8.54
		High	14.53/15.29	9.99/10.92	11.11/12.34	4.96/8.14

Table 3
Relative NR of the wavelet-based ANC implementation referenced to the implementation of ANC without wavelets. All spectrum (20 Hz to 10 kHz), low band (20 Hz to 1 kHz) and high band (1 kHz to 10 kHz).

Mic.	Band	Noise type/NR (dB)			
		Classical music	City noise	Restaurant noise	Pink noise
Monitor	All	1.56	0.75	0.75	0.87
	Low	1.45	0.59	0.60	0.76
	High	3.57	8.23	4.37	1.15
Error (L/R)	All	2.38/2.49	1.49/1.56	2.26/2.34	1.63/1.58
	Low	2.31/2.36	1.39/1.42	2.19/2.18	1.89/1.99
	High	3.17/3.94	8.02/9.22	3.56/5.04	1.49/1.23

wavelets) are more noticeable in the high band. This can be easily visualized through Table 3.

The discussion begins with the analysis of the high band results of the urban and pink noise. These results correspond to the worst and better cases concerning the relative *NR* (see Table 3). The urban noise (which exhibited an *NR* of 8.23 dB) has a high dynamic range. As can be seen through the first spectrogram of Fig. 13, the low band has considerably more intensity than the high band. The implementation without wavelets has one *FxNLMS* module that operates in the entire spectrum. However, the wavelet's implementation has 64 *NLMS* modules (running in parallel), and each one is dedicated to a sub-region of the spectrum. In the *NLMS* algorithm, the step size parameter depends on the signal's energy (see Eq. (4)). Therefore, each one of the *NLMS* has its step size adjusted to the energy of the respective sub-region of the spectrum. In this way, the wavelet's implementation can reduce the entire spectrum more efficiently compared to the implementation without wavelets.

The relative *NR* for the pink noise in the high band was 1.15 dB (see Table 3). In this type of noise, the intensity gradually diminishes with the increase in frequency, as can be seen in the first spectrogram of Fig. 15. The noise's spectral power is more balanced throughout the spectrum, and one unique *NLMS* (as in the case of the implementation without wavelets) achieves a reasonable canceling performance.

The classical and restaurant noises obtained intermediary results in the high band (3.57 dB and 4.37 dB, respectively). In these cases, the noise's spectral power is not so well balanced as in the case of the pink noise and is not as concentrated on the low band as in the case of the city noise.

Wavelet's results

The results for the wavelet's implementation are coherent and show consistent patterns. Notably, the system cancels frequencies below 8 kHz, which can be explained by the wavelength size and its relation to the distance between error microphones and the angle with the secondary loudspeakers' position. The *ANC* system effectively attenuates wavelengths greater than the distance between the error microphones (approximately 4.3 cm), as phase differences (<120°) are not highly discrepant. However, frequencies above 8 kHz are more challenging to reduce due to greater phase differences (>120°). This creates constructive interference at one error microphone position, preventing a fully destructive solution with both secondary loudspeakers.

Eqs. (11) and (12) calculate the *ANC* system's maximum actuation frequency for the specific arrangement. Variables d_{mics}, d_{spks}, and d_{spks_mics} represent the distances between error microphones and secondary loudspeakers. v_{sound} is the speed of sound. Fig. 16 illustrates the geometric scenario used to deduce Eq. (11), considering the wavefront shown by the blue line.

The maximum actuation frequency for the given geometry is 7.3 kHz (room temperature at 30 °C), which is consistent with the information from *FFT* graphics (Figs. 12–15)

Table 4
FPGA resources utilization of the system's implementation (without/with wavelets).

Resource	Utilization			Available	Utilization %		
LUT	8377	/	82 061	218 600	3.83	/	37.54
LUTRAM	4855	/	4 855	70 400	6.90	/	6.90
FF	3683	/	108 629	437 200	0.84	/	24.85
BRAM	11.5	/	365	545	2.11	/	66.97
DSP	20	/	900	900	2.22	/	100.00
IO	26	/	26	250	10.40	/	10.40
BUFG	2	/	3	32	6.25	/	9.38

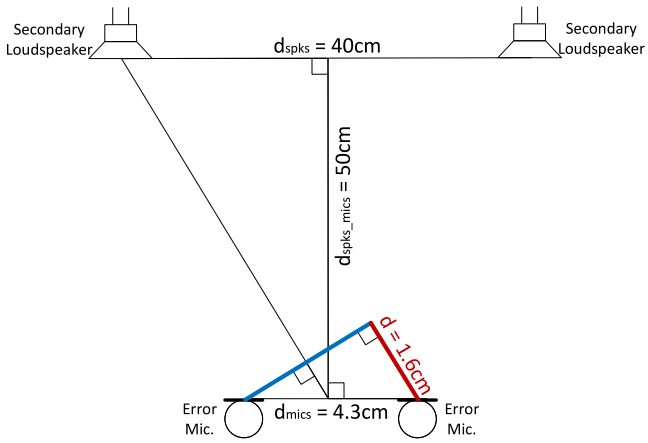


Fig. 16. Geometry used to obtain the formula of Eq. (11). (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

6.2. Logical resources

The *FPGA*'s (*XILINX XC7Z045*) logic resources utilized by the implementations (without wavelets and with wavelets) are presented in Table 4. For the implementation with wavelets, all available *DSPs* were utilized. As mentioned in Section 5.3, three *DSPs* were used for each *FIR* filter of the *NLMS*. One extra *DSP* was used for the calculation of the signal's energy (of the last 1000 samples), and one *DSP* was used for the multiplication of the error signal with the reference signal. Therefore, for each *NLMS* module, and considering the 2×2 dimension of the *MC ANC*, 14 *DSPs* were used (2 *NLMS FIR* filters × 3 + 4 secondary paths × 2). For the *UWPT* modules, no *DSPs* were utilized as explained in Section 5.2. On the other hand, the *FIR* filter modules of the secondary and tertiary paths required one *DSP* each. So, the total number of *DSP*s required for the implementation with wavelets was 14 *DSPs* per *NLMS* × 64 (*NLMS*) + 6 (*FIR* filters of the secondary and tertiary paths), which accounts for 902 *DSPs*. Note that, since the platform only has 900 *DSPs*, the synthesis tool (*Vivado 2018.2*) automatically implemented the extra two *DSPs* on the available logic. For the *MC ANC* without wavelets, the total number of *DSPs* was 20 (14 *DSPs* for the *NLMS* + 6 *FIR* filters of the secondary and tertiary paths).

Both implementations (with and without wavelets) share the same implementation for the *NLMS* and *FIR* filter (of the secondary and tertiary paths). The differences between implementations are the quantity of instantiated *NLMS* modules (which are 64 in the case of the implementation with wavelets and only one for the case of the implementation without wavelets) and the presence of the *UWPT* module (7 modules were instantiated for the wavelet's version, and none were present in the implementation without wavelets).

The number of coefficients $\hat{\boldsymbol{w}}$ in the adaptive filter and the number of coefficients of the *FIR* filters (static filters of the secondary and tertiary paths) were the same in both versions. The same configuration was used in both implementations for a coherent observation of the

impact of the wavelets on the performance of the *MC ANC FxNLMS*.

$$d = cos\left[\frac{\pi}{2} - atan\left(\frac{d_{spks}}{2d_{spks\_mics}}\right)\right] d_{mics} \tag{11}$$

$$f_{max} = \frac{v_{sound}}{3d} \tag{12}$$

7. Conclusion

This article presents a wavelet-based *MC ANC FxNLMS 2 × 2* design, implementation (on *FPGA*), and field test. The noise tests were conducted using four types of complex noises, and the system's performance was compared with the classical implementation of an *MC ANC* system (i.e., the *FxNLMS* without wavelets). Results show that the wavelet-based implementation reduced wideband noises with high dynamic range by at least 8 dB. Comparison with the implementation without wavelets showed that the noise reduction (*NR*) on the low band (20 Hz to 1 kHz) was similar in both implementations. However, depending on the characteristics of the noise, the *NR* of the wavelet-based implementation on the high band (1 kHz to 8 kHz) was superior (e.g., the relative *NR* for the city noise was 8.23 dB, see Table 3). Results also demonstrate that the wavelet-based approach is more effective at canceling broadband noise with high dynamic range (i.e., the spectral power of the noise varies greatly along the whole spectrum). An analysis of the logic resources required by both implementations was also presented.

The presented implementation of the wavelet-based *MC ANC FxNLMS* combines several approaches: (i) the arrangement of the components (speakers and microphones) allows us to anticipate the undesired noise where we want to reduce it; (ii) the use of wavelets allowed for the decomposition of the noise and error signals into multiple straightforward signals; (iii) the *UWPT* has a high output rate (all output decompositions have the same rate as the input signal), making it easier to process by the *FxLMS* algorithm. It also enables the normalization of the noise signal independently in each wavelet decomposition that feeds the *LMS* coefficient update algorithm. That makes all decomposition error signals converge to zero at almost the same rate, regardless of the decomposed signal's power. The use of the Haar wavelet and distinctive implementation approaches allowed us to simplify and make it possible to place an entire *MC ANC* system on an *FPGA* platform.

Based on these findings, it is possible to infer that the Haar wavelet-based *MC ANC FxNLMS* implemented on an *FPGA* is a promising method for reducing undesired ambient noise.

8. Future work

The system is based on injecting an anti-phase signal at the loudspeakers, being the noise canceled at the error microphone's position, but to the listeners' perspective, it is not, as they are on a medium that was not evaluated and the noise and correction evolve with different wavevectors. An improved system implementing a constellation of error microphones and secondary loudspeakers will be able to solve this.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

This work has been supported by FCT – Fundação para a Ciência e Tecnologia, Portugal within the R&D Units Project Scope: UIDB/00319/2020.

The author, Paulo Santos, has been supported by a Portuguese Scholarship from FCT - Fundação para a Ciência e Tecnologia, Portugal and Bosch Car Multimedia, Portugal, under the Advanced Engineering Systems for Industry (AESI) doctoral program (Scholarship ID: PD/BDE/142901/2018).

The author, Eduardo Mendes, has been supported by a Portuguese Scholarship from FCT - Fundação para a Ciência e Tecnologia, Portugal and Bosch Car Multimedia, Portugal, under the Advanced Engineering Systems for Industry (AESI) doctoral program (Scholarship ID: PD/BDE/150499/2019).

Appendix A. Supplementary data

Supplementary material related to this article can be found online at https://doi.org/10.1016/j.micpro.2024.105047.

References

[1] B. Widrow, J. Glover, J. McCool, J. Kaunitz, C. Williams, R. Hearn, J.R. Zeidler, J. Dong, R. Goodlin, Adaptive noise cancelling: Principles and applications, Proc. IEEE 63 (1976) 1692–1716, http://dx.doi.org/10.1109/PROC.1975.10036.

[2] L.Y.L. Ang, Y. Koh, H. Lee, The performance of active noise-canceling headphones in different noise environments, Appl. Acoust. 122 (2017) 16–22, http://dx.doi.org/10.1016/j.apacoust.2017.02.005.

[3] N. Zafeiropoulos, J. Zollner, V. Rajan, State-of-the-art digital road noise cancellation by Harman, ISBN: 978-3-658-20250-7, 2019, pp. 59–74, http://dx.doi.org/10.1007/978-3-658-20251-4.

[4] S. Mallat, A Wavelet Tour of Signal Processing: The Sparse Way, third ed., Academic Press, 2008.

[5] D. Robertson, O. Camps, J. Mayer, W. Gish, Wavelets and electromagnetic power system transients, IEEE Trans. Power Deliv. 11 (2) (1996) 1050–1058, http://dx.doi.org/10.1109/61.489367.

[6] S. Attallah, The wavelet transform-domain LMS algorithm: a more practical approach, IEEE Trans. Circuits Systems II: Analog Digit. Signal Process. 47 (3) (2000) 209–213, http://dx.doi.org/10.1109/82.826747.

[7] Z. Qiu, C.-R. Lee, Z.H. Xu, L.N. Sui, A multi-resolution filtered-x LMS algorithm based on discrete wavelet transform for active noise control, Mech. Syst. Signal Process. 66 (2015) http://dx.doi.org/10.1016/j.ymssp.2015.05.024.

[8] M. Rakhshan, E. Moula, F. Shabani-nia, B. Safarinejadian, S. Khorshidi, Active noise control using wavelet function and network approach, J. Low Freq. Noise Vib. Act. Control 35 (2016) http://dx.doi.org/10.1177/0263092316628260.

[9] M. Akraminia, M.J. Mahjoob, Adaptive feedback active noise control using wavelet frames: simulation and experimental results, J. Vib. Control 22 (7) (2016) 1895–1912, http://dx.doi.org/10.1177/1077546314545835.

[10] L. Luo, J. Sun, B. Huang, A novel feedback active noise control for broadband chaotic noise and random noise, Appl. Acoust. 116 (2017) http://dx.doi.org/10.1016/j.apacoust.2016.09.029.

[11] S. Wen, W.-S. Gan, D. Shi, An improved selective active noise control algorithm based on empirical wavelet transform, in: ICASSP 2020 - 2020 IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP, 2020, pp. 1633–1637, http://dx.doi.org/10.1109/ICASSP40776.2020.9054452.

[12] A. Klemd, B. Klauer, J. Timmermann, S. Delf, A flexible multi-channel feedback FxLMS architecture for FPGA platforms, 2021, pp. 319–326, http://dx.doi.org/10.1109/FPL53798.2021.00063.

[13] D. Shi, W.-S. Gan, J. He, B. Lam, Practical implementation of multichannel filtered-x least mean square algorithm based on the multiple-parallel-branch with folding architecture for large-scale active noise control, IEEE Trans. Very Large Scale Integr. (VLSI) Syst. PP (2019) 1–14, http://dx.doi.org/10.1109/TVLSI.2019.2956524.

[14] D. Mendez, D. Arévalo, D. Patino, E.A. Gerlein, R. Quintana, Parallel architecture of reconfigurable hardware for massive output active noise control, Parallel Process. Lett. 29 (2019) 1950014, http://dx.doi.org/10.1142/S0129626419500142.

[15] B. Widrow, D. Shur, S. Shaffer, On adaptlve inverse control, in: 15th Asilomar Conf. Circuits, Systems, and Components, 1981, pp. 185–189.

[16] B. Widrow, M.E. Hoff, Adaptive switching circuits, in: 1960 IRE WESCON Convention Record, Part 4, Institute of Radio Engineers, New York, 1960, pp. 96–104, URL http://www-isl.stanford.edu/{~}widrow/papers/c1960adaptiveswitching.pdf.

[17] B. Widrow, J. McCool, M. Ball, The complex LMS algorithm, Proc. IEEE 63 (1975) 719–720, http://dx.doi.org/10.1109/PROC.1975.9807.

[18] B. Widrow, D.S. Stearns, Adaptive Signal Processing, Prentice-Hall, Englewood Cliffs, New Jersey, 1985.
[19] S. Haykin, B. Widrow, Least-Mean-Square Adaptive Filters, first ed., Wiley-Interscience, 2003.
[20] S. Haykin, Adaptive Filter Theory, fifth ed., Pearson, Hamilton, Ontario, Canada, 2013.
[21] I. Daubechies, Orthonormal bases of compactly supported wavelets, Commun. Pure Appl. Math. 41 (7) (1988) 909–996, http://dx.doi.org/10.1002/cpa.3160410705, URL https://onlinelibrary.wiley.com/doi/abs/10.1002/cpa.3160410705.
[22] M. Smith, T. Barnwell, A procedure for designing exact reconstruction filter banks for tree-structured subband coders, in: ICASSP '84. IEEE International Conference on Acoustics, Speech, and Signal Processing, Vol. 9, 1984, pp. 421–424, http://dx.doi.org/10.1109/ICASSP.1984.1172486.
[23] M. Vetterli, Filter banks allowing perfect reconstruction, Signal Process. 10 (1986) 219–244.
[24] P. Vaidyanathan, Quadrature mirror filter banks, M-band extensions and perfect-reconstruction techniques, IEEE ASSP Mag. 4 (3) (1987) 4–20, http://dx.doi.org/10.1109/MASSP.1987.1165589.
[25] M. Shensa, The discrete wavelet transform: wedding the a trous and Mallat algorithms, IEEE Trans. Signal Process. 40 (10) (1992) 2464–2482, http://dx.doi.org/10.1109/78.157290.

Paulo Santos has an M.Sc. in Electronics Engineering from the University of Minho, Portugal (2016). He is currently pursuing a Ph.D. degree (University of Minho and Bosch Car Multimedia). His research interests include computational electromagnetism and signal processing.

Eduardo Mendes has an M.Sc. in Electronics Engineering from the University of Minho, Portugal (2017). He is currently pursuing a Ph.D. degree (University of Minho and Bosch Car Multimedia). His interests include Haptics and FPGA.

João Carvalho is currently pursuing a Ph.D. degree (University of Minho). His interests include signal processing and wavelets.

Filipe Serra Alves is currently a Staff Researcher within the Integrated Micro and Nanotechnologies Research Group at INL. His research is focused on the development, design and fabrication of MEMS devices, and the instrumentation and control electronics, both at the discrete and integrated level. He obtained his Ph.D. in the microelectronics research field at the University of Minho - Portugal, in collaboration with the Delft University of Technology - The Netherlands.

José Azevedo is graduated as Mechanical Engineer and Group Manager for automotive sensors. Previously, mechanical designer of automotive sensors and infotainment systems. His interests are Magnetic, Inductive and optical systems.

Jorge Cabral (Member, IEEE) received the Ph.D. degree in microsystems technology from Imperial College London, London, U.K. He is currently an Associate Professor with the University of Minho, Portugal and at CEiiA board of directors.